I.2 Unterstützung für Javadoc in BlueJ

Wenn ein Projekt im Javadoc-Stil kommentiert wurde, dann bietet BlueJ Unterstützung bei der Generierung der kompletten HTML-Dokumentation. Selektieren Sie im Hauptfenster den Menüeintrag WERKZEUGE/DOKUMENTATION ERZEUGEN und die Dokumentation wird generiert und in einem Browserfenster angezeigt.

Im BlueJ-Editor kann von der Quelltextsicht auf die Dokumentationssicht umgeschaltet werden, indem die Option *Implementierung* im Fenster oben rechts auf *Schnittstelle* umgestellt wird (Abbildung I.1). Dies ermöglicht eine schnelle Voransicht der Dokumentation, die jedoch keine Referenzen auf Superklassen oder benutzte Klassen enthält.

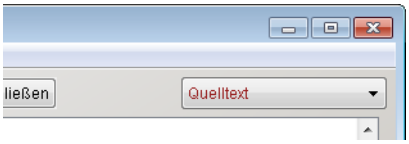

Abbildung I.1
Die Implementierung/Schnittstelle-Option im BlueJ-Editor.

Weitergehende Informationen sind verfügbar unter:

http://www.oracle.com/technetwork/java/javase/documentation/index-137868.html

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:06 Uhr

ANHANG

J

Quelltextkonventionen

J.1 Benennung

J.1.1 Sprechende Namen verwenden

Benutzen Sie sprechende Namen für alle Bezeichner (Klassen-, Variablen- und Methodennamen). Vermeiden Sie Mehrdeutigkeiten. Vermeiden Sie Abkürzungen. Einfache verändernde Methoden sollten in der Form *setzeIrgendetwas(…)* benannt werden. Einfache sondierende Methoden sollten in der Form *gibIrgendetwas(…)* benannt werden. Sondierende Methoden mit einem booleschen Ergebnis sind oft mit *istIrgendetwas(…)* benannt, beispielsweise `istLeer()`.

J.1.2 Klassennamen beginnen mit einem Großbuchstaben

J.1.3 Klassennamen sind Hauptwörter im Singular

J.1.4 Methoden und Variablennamen beginnen mit einem Kleinbuchstaben

Benutzen Sie für alle drei – für Klassen-, Methoden- und Variablennamen – Großbuchstabe am Anfang eines neuen Wortes nach dem ersten Wort, um zusammengesetzte Namen lesbarer zu machen, etwa `anzahlEintraege`.

J.1.5 Konstanten werden in GROSSBUCHSTABEN geschrieben

Verwenden Sie Unterstriche, um zusammengesetzte Konstantennamen lesbarer zu gestalten:

`OBERE_GRENZE`[1]

1 Anmerkung des Übersetzers: Im Deutschen ist dies aufgrund zusammengesetzter Hauptwörter nicht so häufig notwendig wie im Englischen.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:06 Uhr

J.2 Layout

J.2.1 Eine Stufe der Einrückung besteht aus vier (oder drei) Leerzeichen

J.2.2 Alle Anweisungen innerhalb eines Blocks sind um eine Stufe eingerückt

J.2.3 Klammern für Klassen und Methoden stehen allein auf einer Zeile

Die geschweiften Klammern für den Block einer Klasse oder einer Methode stehen jeweils auf einer eigenen Zeile und befinden sich auf derselben Einrückungsstufe. Beispielsweise:

```
public int gibAlter()
{
    Anweisungen
}
```

J.2.4 Für alle anderen Blöcke steht die öffnende Klammer am Ende einer Zeile

Alle anderen Blöcke beginnen mit einer geschweiften Klammer am Ende der Zeile mit dem einleitenden Schlüsselwort für den Block. Die schließende Klammer steht auf einer eigenen Zeile und ist genauso weit eingerückt wie das einleitende Schlüsselwort für den Block. Beispielsweise:

```
while(Bedingung) {
    Anweisungen
}
if(Bedingung) {
    Anweisungen
}
else {
    Anweisungen
}
```

J.2.5 In Kontrollstrukturen immer Klammern verwenden

Verwenden Sie immer geschweifte Klammern in `if`-Anweisungen und Schleifen, auch wenn der Rumpf nur aus einer einzelnen Anweisung besteht.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:06 Uhr

J.2.6 Vor der öffnenden Klammer für den Block einer Kontrollstruktur steht ein Leerzeichen

J.2.7 Operatoren links und rechts mit einem Leerzeichen absetzen

J.2.8 Immer eine Leerzeile zwischen Methoden (und Konstruktoren)

Benutzen Sie Leerzeilen, um logische Abschnitte im Quelltext voneinander abzusetzen; also mindestens zwischen Methoden immer eine Leerzeile, aber auch zwischen logischen Abschnitten innerhalb einer Methode.

J.3 Dokumentation

J.3.1 Jede Klasse wird mit einem Klassenkommentar eingeleitet

Der Klassenkommentar enthält mindestens

- eine allgemeine Beschreibung der Klasse
- Namen des/der Autoren
- eine Versionsnummer.

Jede Person, die zur Klasse beigetragen hat, sollte als Autor aufgeführt sein oder in anderer Form erwähnt sein.

Eine Versionsnummer kann eine einfache Nummer oder ein Datum sein, sie kann aber auch in einem anderen Format vorliegen. Wichtig ist, dass ein Leser an ihr erkennen kann, dass zwei Versionen sich unterscheiden und welche von ihnen die neuere ist.

J.3.2 Jede Methode hat einen Methodenkommentar

J.3.3 Kommentare sind im Javadoc-Format

Klassen- und Methodenkommentare müssen von Javadoc erkannt werden können. Anders gesagt: Sie sollten mit der Zeichenfolge `/**` beginnen.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:06 Uhr

J.3.4 Implementierungskommentare nur falls notwendig

Kommentare in den Anweisungsteilen einer Klasse sollten eingefügt werden, wenn der Quelltext nicht leicht zu verstehen ist (und es sollte eher versucht werden, den Quelltext so leicht verständlich wie möglich zu machen). Kommentieren Sie keine offensichtlichen Anweisungen – gehen Sie davon aus, dass Ihr Leser Java versteht.

J.4 Restriktionen bei der Sprachbenutzung

J.4.1 Reihenfolge der Deklarationen: Datenfelder, Konstruktoren, Methoden

Die Elemente einer Klassendefinition erscheinen (falls vorhanden) in folgender Reihenfolge: Paketzugehörigkeit; **`import`**-Anweisungen; Klassenkommentar; Klassenkopf; Definitionen der Datenfelder; Konstruktoren; Methoden; innere Klassen.

J.4.2 Datenfelder dürfen nicht als public deklariert werden (Ausnahme: konstante Datenfelder)

J.4.3 Immer einen Zugriffsmodifikator benutzen

Deklarieren Sie alle Datenfelder und Methoden entweder als **`private`**, **`public`** oder **`protected`**. Benutzen Sie nicht den Default ohne Schlüsselwort (Paket-privater Zugriff).

J.4.4 Klassen einzeln importieren

Import-Anweisungen, die jede Klasse einzeln benennen, sind dem Import von ganzen Paketen vorzuziehen. Beispielsweise

```
import java.util.ArrayList;
import java.util.HashSet;
```

ist besser als

```
import java.util.*;
```

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:06 Uhr

J.4.5 Immer einen Konstruktor deklarieren (auch wenn der Rumpf leer ist)

J.4.6 Immer einen expliziten super-Aufruf im Konstruktor einfügen

Verlassen Sie sich im Konstruktor einer Subklasse nicht darauf, dass ein Aufruf des Konstruktors der Superklasse automatisch eingefügt wird. Fügen Sie den Aufruf `super()` explizit ein, auch wenn es ohne ginge.

J.4.7 Alle Datenfelder im Konstruktor initialisieren

Einschließlich Zuweisungen von Standardwerten. Dadurch wird es für einen Leser deutlich, dass die korrekte Initialisierung aller Datenfelder beachtet wurde.

J.5 Programmiermuster

J.5.1 Iteratoren für Sammlungen benutzen

Benutzen Sie eine `for-each`-Schleife, um über eine Sammlung zu iterieren. Wenn die Sammlung während der Iteration geändert werden muss, verwenden Sie einen `Iterator`, keinen `int`-Index.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:06 Uhr

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:06 Uhr

ANHANG

K

Wichtige Bibliotheksklassen

Die Java-Plattform beinhaltet eine große Anzahl von Bibliotheken, die ein weites Spektrum an Programmieraufgaben unterstützen.

In diesem Anhang geben wir eine kurze Übersicht einiger Klassen und Interfaces aus den wichtigsten Paketen des Java-API. Ein kompetenter Java-Programmierer sollte mit den meisten von ihnen vertraut sein. Dieser Anhang gibt lediglich einen Überblick und sollte in Verbindung mit der vollständigen API-Dokumentation gelesen werden.

K.1 Das Paket java.lang

Die Klassen und Interfaces im Paket `java.lang` sind grundlegend für die Sprache Java. Deshalb ist dieses Paket implizit und automatisch in jeder Klassendefinition importiert.

Tabelle K.1
Paket `java.lang` – Kurzdarstellung der wichtigsten Klassen.

Klasse / Interface	Beschreibung
Interface `Comparable`	Die Implementierung dieses Interface ermöglicht den Vergleich und die Sortierung von Objekten der implementierenden Klasse. Statische Hilfsmethoden wie `Arrays.sort` und `Collections.sort` können für Objekte von Typen, die `Comparable` implementieren, zum effizienten Sortieren genutzt werden.
Interface `Runnable`	Ein Interface, das keine Parameter hat und kein Ergebnis zurückgibt, es wird als Typ für Lambda-Ausdrücke mit diesen Eigenschaften verwendet.
Klasse `Math`	Die Klasse `Math` enthält ausschließlich statische Datenfelder und Methoden. Die Werte der mathematischen Konstanten `e` und sind hier definiert, zusammen mit den trigonometrischen und anderen Funktionen, wie beispielsweise `abs` (Betrag), `min`, `max` und `sqrt` (Quadratwurzel).
Klasse `Object`	Alle Klassen haben `Object` als die Wurzelklasse ihrer eigenen Klassenhierarchie. Von dieser Klasse erben alle Objekte Standardimplementierungen von so wichtigen Methoden wie `equals` und `toString`. Weitere wichtige Methoden in dieser Klasse sind `clone` und `hashCode`.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:06 Uhr